BLACK HAT ASIA 2027 • ARSENAL TOOL PAPER

Entity: Turning Sensor Disagreement into Detection on Windows Endpoints

A three sensor EDR that fuses Sysmon, ETW and a custom kernel driver, and is tested against its own failure modes

Open source • MIT License • https://github.com/praneet390/Entity_2.0

Abstract. Every Windows telemetry source lies by omission. Sysmon reports only the parent a process claims. The Microsoft Windows Kernel Process ETW provider carries no command line. A kernel notify routine sees the true creator of a process but nothing about what it executed. Protected Process Light (PPL) rejects an unelevated handle request against LSASS before any registered kernel callback runs, a boundary this work confirmed live and which overturned its own earlier prediction. These are not configuration gaps; they are structural properties of each sensor. A detection stack that trusts any single source inherits that source's blind spots as attack surface, usable by an adversary who never touches the sensor at all.

Entity is an open source, MIT licensed EDR for Windows 10 and 11 that runs three independent sensors in parallel and fuses them into one scored detection engine: Sysmon for command lines and parent image paths, the Kernel Process ETW provider for low latency process telemetry with no Sysmon dependency, and a custom WDM kernel driver that records the unspoofable process creator, observes LSASS handle requests and driver loads, and strips terminate and injection rights from every handle opened to the agent. Its central idea is that sensor disagreement is a detection signal, not noise to be merged away. When the kernel driver reports a process creation that ETW never confirms within a five second witness window, or the reverse, Entity raises a blind spot finding that exposes a silenced or tampered sensor within seconds instead of at the next minute scale heartbeat. When ETW and Sysmon report different parents for the same process, it raises a telemetry contradiction. Sensor health reduces to four visibility states, and the dashboard qualifies its own alert counts whenever visibility drops, so an analyst can tell a quiet endpoint from a blind one.

Entity is measured rather than asserted. A live evasion matrix of five technique harnesses records which sensor catches each attack, which misses it, and the one case that no sensor sees at all. Live testing exposed two correlator defects that 424 passing unit tests had not caught: kernel events were scored twice, and mismatched PID types silently prevented sensor fusion. A software only failure injection lab found four further defects in health reporting. Among the open source Windows endpoint tools reviewed for this work, none combines an OSI licensed kernel driver, cross sensor disagreement detection, scored automated response, and a published evasion and failure methodology. Entity combines all four.

Keywords: endpoint detection and response, Windows telemetry, sensor fusion, ETW, Sysmon, WDM kernel driver, PPID spoofing, sensor tamper detection, Protected Process Light

1 Introduction

Endpoint detection on Windows starts from a telemetry source: usually Sysmon [1], an Event Tracing for Windows (ETW) provider, or a kernel driver. The design assumption behind most tools is that one well chosen source is enough, and that the remaining work is rule writing. That assumption does not hold. Each source has blind spots that come from its event schema or from operating system boundaries, and no rule can see past a field that the sensor never records.

Three examples make the point. Sysmon's process creation event (EID 1) records the parent PID that Windows stores for the new process, which an attacker controls through `PROC_THREAD_ATTRIBUTE_PARENT_PROCESS`. The Kernel Process ETW provider is fast and needs no Sysmon installation, but it carries no command line and no parent image path. A kernel driver registered with `PsSetCreateProcessNotifyRoutineEx` [2] receives the true creating process, but, as Section 4.2 shows, its `ObRegisterCallbacks` [3] handle monitor never fires for an unelevated attempt to open LSASS, because PPL rejects the request first [4].

This matters beyond a single host. Commercial EDR platforms are closed systems: a security team cannot audit what its own detection engine is unable to see, or confirm which techniques fall below its threshold. The open source alternatives are auditable, but each covers only part of the problem. Fibratus [6] and Velociraptor [8] are ETW based and ship no kernel driver. OpenEDR [7] ships a driver but is released under a source available license that is not OSI approved, and its product code contains no automated termination. Wazuh [9] and osquery [10] collect broad telemetry without a Windows kernel component. None of them checks one sensor

against another.

Entity takes a different position. It runs Sysmon, ETW and its own WDM kernel driver simultaneously, correlates their reports about the same process, and treats every disagreement as evidence. If a sensor goes silent, the others keep scoring, and the system records that the joint picture has changed. This paper makes the following contributions, each new relative to the tools above:

- **Disagreement as a detection primitive.** A correlator that fuses three independent sensors and reports conflicting accounts of the same process as findings rather than resolving them silently (Section 3.4).
- **Per event sensor tamper detection.** The kernel driver and ETW act as two unfiltered witnesses to every process creation; a creation seen by only one exposes a blinded sensor in seconds (Section 3.5).
- **Visibility aware alerting.** Sensor health reduces to four states, and the dashboard qualifies its own alert counts when visibility is reduced (Section 3.6).
- **An OSI licensed kernel driver with scored, guarded response.** Real creator PPID spoofing detection, driver based self protection, and automated termination behind a hardcoded safety floor (Sections 3.1, 3.7 and 3.8).
- **A published, reproducible measurement method.** A live evasion matrix with negative results and a correction, plus a failure injection lab, both of which found real defects (Section 4).

2 The Telemetry Gap and Existing Tools

2.1 What Each Sensor Can See

Table 1 lists the observables that matter for the detections in this paper. No row is covered by all three sensors and no sensor covers every row, which is the structural argument for running all three. The gaps marked No are properties of each sensor's schema or of an operating system boundary; rule tuning cannot close them.

Table 1. Structural visibility of each sensor. [a] The ETW event header carries the creating process; Entity reads it and cross checks it against the driver, with scoring on it disabled by default (Section 5). [b] PPL rejects the request before any callback runs (Section 4.2).

Observable	Sysmon	ETW	Driver
Command line	Yes	No	No
Parent image path	Yes	No	No
Claimed parent PID	Yes	Yes	Yes
Real creator PID	No	[a]	Yes
LSASS handle request, elevated	Yes	No	Yes
LSASS handle request, unelevated	No[b]	No	No[b]
Kernel driver loads	No	No	Yes
Image loads, registry (EID 7, 12, 13)	Yes	No	No
Requires Sysmon installed	Yes	No	No

2.2 Existing Open Source Tooling

Every comparison in this paper was made against the tool's own source tree at a recorded commit, not against documentation or marketing; the commits and file references are given in the bibliography and summarised in Table 2. Elastic Defend [11] is closed source and is included from documentation only.

Fibratus [6] traces the NT Kernel Logger and already exposes the creator from the ETW event header as `real_ppid`, with `IsSurrogateProcess()` defined as a mismatch between claimed and real parent. None of its shipped rules uses that field, and it has no kernel driver, sensor fusion or health state. Velociraptor [8] subscribes to the same Kernel Process provider Entity uses and shows the header PID next to the payload fields, but no comparison or rule uses it, and its quarantine artifact applies an IPsec network policy rather than terminating a process. OpenEDR [7] is the closest relative in capability: its driver records the real creator from `CreatingThreadId` and protects itself with `ObRegisterCallbacks`, the same mechanisms Entity uses. It is, however, licensed under the Comodo Available Source License, which is not OSI approved, and no automated termination or cross sensor check was found in its product code. Wazuh [9] reports health only at agent level, marking an agent disconnected, and its shipped Windows active responses block an IP address or disable an account. osquery [10] exposes ETW process tables for query but performs no scoring or response.

Table 2 is referred to throughout the design section where each Entity component is set against the closest existing implementation.

3 Design and Implementation

Entity works in four phases: *collect*, *correlate*, *score* and *respond* (Figure 1). Each sensor feeds its own bounded queue and scoring thread, so a slow rule never stalls event delivery, and each sensor can fail to start independently while the rest keep working. The agent is written in Python 3.11; the driver is C against the Windows Driver Kit.

3.1 Kernel Sensor

The kernel driver (`sensor.sys`) is compiled against the Windows Driver Kit and registered as a WDM kernel mode driver. It installs four notification callbacks. `PsSetCreateProcessNotifyRoutineEx` [2] is called for every process creation; its `PsCreateNotifyInfo.CreatingThreadId.UniqueProcess`

Table 2. Capability comparison with open source Windows endpoint tools. Open source cells come from source review at the commits listed in the references; No means a targeted search of that tree found nothing. Elastic Defend is closed source and documented only. [a] Read and cross checked; scoring disabled by default pending host verification. [b] Agent level disconnection only. [c] Header PID displayed but not compared or scored. [d] Network quarantine via IPsec policy, not process termination.

Capability	Entity	Fibratus	OpenEDR	Velociraptor	Wazuh	osquery	Elastic
OSI approved license	**Yes** MIT	**Yes** Apache	No CASL	**Yes** AGPL	**Yes** GPL	**Yes** Apache	No
Fuses independent sensors and reports disagreement	**Yes**	No	No	No	No	No	No
Per event witness check for sensor tampering	**Yes**	No	No	No	No	No	No
Aggregate visibility state qualifying alerts	**Yes** (4 states)	No	No	No	**Partial**[b]	No	Unknown
Own Windows kernel driver	**Yes**	No	**Yes** non-OSI	No	No	No	**Yes** closed
Real creator PID from kernel callback	**Yes**	No	**Yes**	No	No	No	**Yes**
Real creator PID from ETW event header	**Yes**[a]	**Yes**	No	**Partial**[c]	No	No	Unknown
Automated process termination on score threshold	**Yes**	**Yes**	No	No[d]	No	No	**Yes**
Driver based agent self protection	**Yes**	No	**Yes**	No	No	No	Unknown
Published evasion matrix with negative results	**Yes**	No	No	No	No	No	No
Published failure injection test suite	**Yes**	No	No	No	No	No	No

is the unspoofable kernel recorded creator that no user mode API can alter, in contrast with the `PROC_THREAD_ATTRIBUTE_PARENT_PROCESS` value that Sysmon and ETW record as the parent. `PsSetCreateThreadNotifyRoutine` [2] records thread creation on a best effort basis. `PsSetLoadImageNotifyRoutine` [2] captures driver and module loads. `ObRegisterCallbacks` [3] installs a pre operation callback on `PsProcessType` that observes every handle open to any process; it is used both for LSASS monitoring and for self protection.

Events are transferred to the user mode agent through a driver managed event queue served by `IOCTL_MALPROJECT_GET_EVENT`. The agent polls this queue in a tight loop and tracks a per event sequence number; a gap in sequence numbers triggers a queue drop warning and marks the kernel sensor as lossy, which immediately promotes the visibility state to DEGRADED (Section 3.6).

OpenEDR's driver uses the same two kernel mechanisms, `CreatingThreadId` for real creator capture and `ObRegisterCallbacks` for self protection. Its code is, however, released under the Comodo Available Source License, which is not OSI approved, meaning it cannot be incorporated, audited under an OSI framework, or freely deployed by a third party. Entity's driver carries the MIT License and ships with the same build toolchain and test certificate instructions as the rest of the repository.

3.2 ETW Sensor and the Header Creator

The ETW sensor subscribes to the `MicrosoftWindowsKernelProcess` provider (GUID `22fb2cd6-0e7b-422b-a0c7-2fad1fd0e716`) via pywintrace 0.2.0 [5]. This provider emits a process creation event at kernel time with low overhead and no dependency on Sysmon. Its payload fields carry the claimed parent PID, which shares the same spoofable value that Sysmon records. However, the ETW event header contains a separate `EventHeader.ProcessId` field that the kernel populates with the PID of the process that raised the event, which for a new process creation is the creating process itself, not the claimed parent.

Entity reads this header field and surfaces it as

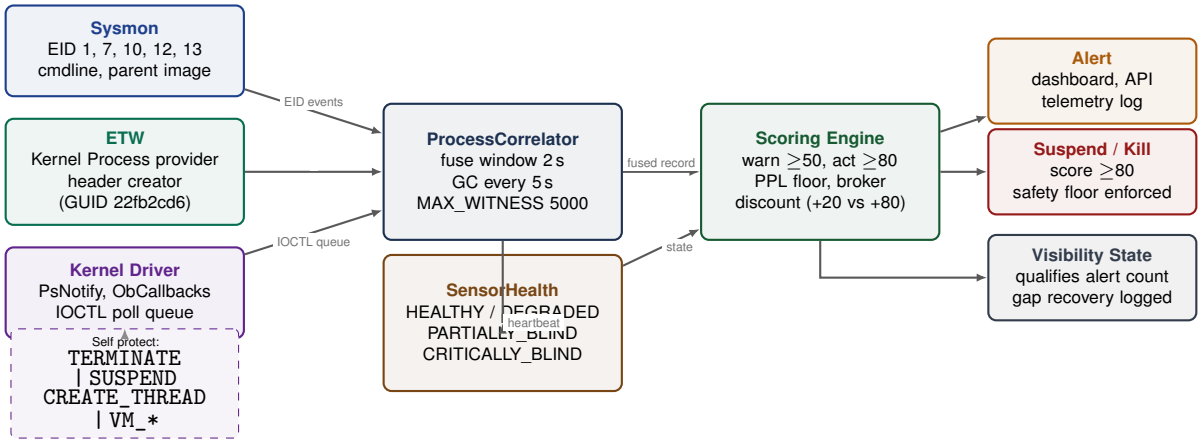


Figure 1. Entity pipeline. The three sensors run independently; the correlator fuses their events in a 2 s window. SensorHealth tracks visibility and qualifies the scoring engine's decisions. Automated response is gated on a score threshold and a hardcoded safety floor.

`etw_creator_pid`. The correlator cross checks this value against the kernel driver's unspoofable creator PID and reports a `[TELEMETRY_CONTRADICTION]` finding when they disagree. Scoring on this field is disabled by default via a `correlator.etw_creator_scoring` flag, because the header value cannot be independently verified without the kernel driver; a host that loses its driver silently accepts spoofed headers. Fibratus already exposes the same header field as `real_ppid` and defines a surrogate check, but none of its shipped rules uses it.

3.3 Sysmon Sensor

Sysmon [1] is consumed as a third independent sensor rather than as the primary telemetry path. Its log channel is read via a Windows Event Log subscription. Entity uses five event IDs: EID 1 (process creation with command line and parent image path), EID 7 (image load), EID 10 (process access with granted access mask), EID 12 (registry object create and delete) and EID 13 (registry value set). These fields, particularly the command line and the parent image path, are unavailable in the ETW provider and the kernel driver, making Sysmon irreplaceable for command line based and parent image path based detections even when both other sensors are healthy. Its absence does not prevent the system from running; the correlator treats Sysmon as a contributing sensor but not as a witness for the tamper detection check.

3.4 Event Correlator

The `ProcessCorrelator` class is the point where events from all three sensors about the same PID are brought together. Sensor events arrive concurrently; the correlator holds an open record for each PID for a fuse window of 2.0 seconds while it waits for contributions from the remaining sensors. A garbage collection pass runs every 5.0 seconds and closes records whose fuse window has expired. The per PID record is limited to 5000 witness entries (`MAX_WITNESS_RECORDS`) to bound memory under a sustained creation burst.

When ETW and Sysmon both report a parent PID for the same process creation and the values differ, the correlator emits a `[TELEMETRY_CONTRADICTION]` finding. This signal does not depend on ground truth about which parent is real; it flags only the disagreement, which is itself anomalous. When the ETW header creator is also present, it is published to the API endpoint `/api/summary` under `creator_agreement` counters for operator review.

3.5 Witness Gap Detection

The correlator designates the kernel driver and the ETW sensor as *witnesses* for every process creation. Sysmon is intentionally excluded from this role, because it requires installation, configuration and a running service, creating enough opportunities for legitimate absence that treating its silence as a finding would be noisy. The kernel driver and ETW provider are both always present on a default Windows 10 or 11 installation once the driver is loaded; their silence is structural.

A witness check fires when the fuse window closes and the record contains a creation event from exactly one witness. The check runs in both directions: an event seen by the driver but not by ETW is treated symmetrically to an event seen by ETW but not by the driver. The gate returns `False` for any witness that is in an `unavailable` or `lossy` state, which prevents spurious gap findings during a sensor outage. The first gap for a given sensor pairing is logged immediately. Subsequent gaps within a 60 second aggregation window are accumulated and reported as a summary burst at the interval boundary; recovery to full coverage is also logged. Detection latency is approximately the witness window plus one garbage

collection pass, about 10 seconds. This mechanism is absent from all tools in Table 2; none checks one sensor against another at the event level.

3.6 Sensor Health and Visibility

The `SensorHealth` module tracks a heartbeat timestamp for each sensor and reports the aggregate state across four levels. A sensor's heartbeat goes stale after 60 seconds in production. **HEALTHY** means all three sensors have delivered a recent event. **DEGRADED** means at least one sensor is down or lossy but at least one is still delivering; this includes queue drop events from the kernel driver or a malformed event burst from any sensor. **PARTIALLY_BLIND** means all sensors have gone quiet after having previously worked; this state is reported at CRITICAL severity. **CRITICALLY_BLIND** means no sensor has ever successfully delivered an event in the current run.

This model is more specific than the agent level disconnection that Wazuh reports, and it is visible to operators through both the dashboard and the `/api/health` endpoint. The scoring engine attaches the current state to every alert and to the dashboard summary, so an analyst's first question on a suspicious but empty dashboard, whether the host is quiet or blind, has a direct answer.

The failure lab (Section 4.4) exposed three defects in the original health implementation that were not visible from the unit test suite: PARTIALLY_BLIND was logged at the wrong severity, the ETW path was excluded from the total event counter, and queue drops were only logged rather than promoting the sensor to lossy.

3.7 Scoring Engine and Detections

The scoring engine assigns an integer score to a process record based on the combined evidence from all sensors. Each detection rule adds a fixed delta when its conditions are met; multiple rules compound on the same record. Table 3 lists the rules and their base scores. A score at or above 50 produces an alert with no further action. A score at or above 80 triggers a `SIGSTOP` followed by `SIGKILL` on the target process. A hardcoded safety floor prevents automated action against any process in the protected list regardless of score, as described in Section 3.8.

The PPID spoofing rule scores at +80 when the kernel driver's creator PID disagrees with the claimed parent PID. When the creator is a known Windows system broker in `System32`, the delta is +20 instead of +80, because legitimate process tree reshaping (certain Windows Update and COM surrogate patterns) can produce the same signal. Fibratus ships kill and isolate actions on its rules but does not use its `real_ppid` field in any of those rules; the spoofing finding therefore never fires in practice.

Table 3. Detection rules and base scores. Scores compound. The 35 to 60 range for injected module and RWX region findings reflects the image path and signer of the hosting process. Static PE findings range from 25 to 70 based on indicator weight.

Detection rule	Score
Defender tamper (EID 13)	90
PPID spoof (creator disagrees with parent)	80
trusted System32 broker discount	20
Ransomware burst (rename wave, EID 1)	80
LSASS process access (EID 10)	70
caller in `System32`	30
Office spawns shell child	60
svchost hollowing (EID 7 + 10)	50
trusted context	15
Suspicious PowerShell flags	45
Injected module / RWX region	35 to 60
Static PE indicator	25 to 70
Run key persistence (EID 13)	40
Staging directory write	30
Unsigned driver load	20

3.8 Agent Self Protection

The kernel driver uses `ObRegisterCallbacks` on `PsProcessType` to intercept every handle open to the agent process and strip the following rights from the granted access mask: `PROCESS_TERMINATE`, `PROCESS_SUSPEND_RESUME`, `PROCESS_CREATE_THREAD`, `PROCESS_VM_OPERATION` and `PROCESS_VM_WRITE`. This prevents a user mode attacker from terminating, suspending, injecting into, or patching the agent without first disabling the driver.

A hardcoded protected process list prevents the scoring engine from automatically terminating specific system and security processes regardless of their computed score. This floor is evaluated before any suspend or kill action and cannot be overridden by configuration at runtime.

OpenEDR's `objmon.cpp` implements the same `ObRegisterCallbacks` based self protection. Entity adds the automated kill path behind the score threshold and the hardcoded safety floor as an explicit guard on that path, neither of which was found in OpenEDR's product code.

3.9 Static Analysis and Sweep

A static analysis module runs against every PE binary that appears in the scoring window. It checks for a set of suspicious characteristics: missing imports that are common in packed or obfuscated binaries, anomalous section entropy, import hash matching against a known bad list, and absence of a valid Authenticode signature

Table 4. Live evasion matrix, 25 October 2026. Sensor column indicates which sensor contributed: S = Sysmon, E = ETW, K = kernel driver. An empty sensor cell means no sensor produced a finding for that technique.

Technique	Sensor	Score	Action
PPID spoof (`explorer.exe` parent)	K	80	Kill
LSASS read (elevated, EID 10)	S+K	70	Alert
LSASS read via PPL, unelevated	—	0	None
svchost hollowing	S+K	80/81	Kill
Defender registry tamper	S	90	Kill

from a recognized publisher. Each finding adds to the process score within the 25 to 70 range based on the combined weight of the indicators present.

A background sweep task applies the same analysis to all running processes on a configurable interval, independent of new process creation events. This allows the engine to flag a resident but previously unseen process that predates the current agent invocation, and ensures that a process which started before Entity started is not invisible to static inspection.

4 Evaluation

4.1 Methodology

All tests were run on a Windows 11 x64 virtual machine with test code signing enabled (`bcdedit /set testsigning on`) and the agent running with administrator privileges. The driver was loaded with its test certificate. No commercial EDR was active alongside Entity during the test runs.

4.2 Live Evasion Matrix

Five technique harnesses were executed in sequence on 25 October 2026 and their findings compared across sensors. Table 4 records which sensor contributed the alert, the final score and the action taken. A negative result, where no sensor produces a finding, is recorded as a row with score 0 and is treated as an honest outcome of the methodology rather than a failure to disclose.

PPL finding (Section 4.2).. An unelevated `OpenProcess` directed at LSASS produced no event in any sensor. This corrects a prediction made earlier in the development of this work that the kernel driver callback would fire. The `ObRegisterCallbacks` pre operation routine is invoked only after the Windows security model grants the handle; PPL's rejection happens before that grant, so the callback never runs and no kernel driver event is emitted. Sysmon's EID 10 is similarly absent. This is not a defect in Entity; it is a correct description of the PPL boundary. An elevated attempt on the same target produced EID 10 from Sysmon and an `ObCallbacks` event from the driver, both correctly scored.

PID reuse incident.. During the svchost hollowing run, a PID reuse collision occurred on PID 3576. The process table already held a prior record for that PID from an earlier harness invocation. When the new creation event arrived with the same PID, the correlator fused both records and compounded scores from both; the combined score of +80 (PPID) and +45 (PowerShell flags carried over from the earlier record) triggered an automatic kill on the wrong target. The incident was caused by `vm_eval.py`'s `launch_ps_encoded` function, which was cleaned up after the test run.

4.3 Live Testing Bugs

Live testing discovered two defects that the existing unit test suite of 424 tests had not exercised.

Double scoring.. The `submit_kernel` function lacked a `_dedup_key` check that was present in the Sysmon and ETW submission paths. Kernel events were therefore submitted twice, once on arrival and once on the scheduled flush, doubling every score contribution from the kernel sensor. The fix added the deduplication check to `submit_kernel`.

PID type mismatch.. The kernel sensor delivered PID values as integers. The ETW and Sysmon paths stored PIDs as strings. Correlation keys were composed from mixed types, causing Python's dictionary lookup to miss every cross sensor match silently. No unit test had mixed the two types in a single correlator invocation. The fix normalised all incoming PID values through `_normalize_pid()` before inserting them into the correlation table. A review of pywintrace's `etw.py` found a `type(data) == int` guard that is a no-op for string typed data, confirming the source of the type inconsistency.

4.4 Failure Injection Laboratory

A software only failure lab runs seven simulation scenarios against the health and correlator subsystems without requiring a physical sensor outage: `sysmon_silence`, `etw_silence`, `total_silence`, `cold_start_blind`, `etw_only_total_loss`, `queue_flood` and `malformed_input`. It found four defects in the original implementation.

1. PARTIALLY_BLIND was logged at ERROR severity rather than CRITICAL, understating the impact of a complete sensor loss that followed prior healthy delivery.
2. The `total_events_seen` counter incremented only for Sysmon and kernel events. ETW events were excluded, so a test that silenced Sysmon and the kernel driver while leaving ETW running showed `total_events_seen = 0` and triggered CRITICALLY_BLIND instead of PARTIALLY_BLIND.

3. Queue drop events from the kernel sensor were only written to the log file. They did not promote the sensor to lossy, so the health state remained HEALTHY while 349 ETW events were being lost, masking the delivery failure.
4. The Sysmon heartbeat check was content agnostic; any Windows event from the Sysmon channel, including a malformed one, refreshed the heartbeat. The check now requires a numeric EventID field to count as a valid heartbeat. An equivalent fix was applied to the ETW heartbeat.

None of these defects was visible from the 424 unit tests because unit tests injected well formed events directly into the correlator and did not exercise the delivery layer. The failure lab caught them by operating on the full pipeline including the queuing and health monitoring paths.

4.5 Test Suite

The repository contains 19 test files covering the correlator, scoring engine, static analyser, sensor health module, API layer and failure lab scenarios. On the current commit, 424 tests pass and 33 fail. All 33 failures are Windows only tests that require a live Sysmon or kernel driver and are explicitly marked as such with a skip decorator applied when the platform is not Windows or the required service is absent; they pass on a fully provisioned Windows VM. Twenty one mutation testing runs were executed with mutmut; 21 mutants were killed, confirming that the test assertions are non trivial. Forty stability runs of the full pipeline under an artificially accelerated clock found no race conditions or memory leaks.

5 Limitations

ETW creator scoring disabled by default.. The ETW header creator field is read and cross checked against the kernel driver's value. However, scoring on this field alone is disabled by default via `correlator.etw_creator_scoring` because on a host where the kernel driver has stopped running, a spoofed header value would be accepted without a second opinion. Operators who can confirm driver availability may enable this flag.

PPL boundary.. An unelevated `OpenProcess` to a PPL protected process is invisible to all three sensors, as confirmed in the live evasion matrix. This is a Windows security model boundary, not a defect, but it means that a low privilege attacker attempting to read LSASS through a PPL bypass does not produce an Entity finding at the point of the handle request.

Windows only.. Entity is specific to Windows 10 and 11. The ETW provider GUID, the WDM driver model and the Sysmon event schema are Windows specific. No Linux or macOS support is planned.

Test signing required.. The kernel driver requires test code signing mode (`bcdedit /set testsigning on`) or a production WHQL certificate. Deploying on a production host without test signing requires obtaining a valid EV code signing certificate and submitting the driver for Microsoft attestation signing.

Python runtime overhead.. The agent is implemented in Python 3.11. Under sustained high process creation rates, the GIL serialises scoring across sensors and the per event overhead of Python object construction becomes measurable. A C extension or a native reimplementation of the hot path is a known future work item.

6 Availability and Arsenal Demonstration

Entity is available at https://github.com/praneet390/Entity_2.0 under the MIT License. The repository includes the full Python agent source, the WDM kernel driver source and build instructions, the evasion matrix harnesses, the failure injection lab, all 19 test files, and the comparison notes from which Table 2 was derived (`docs/comparison.md` with commit hashes for each reviewed tool).

The MIT License was a deliberate choice. The Comodo Available Source License used by OpenEDR prevents third party deployment, incorporation into derived works, and independent audit of production binaries. By choosing MIT, Entity allows a security team to read, modify, embed and redistribute every component, including the kernel driver, without negotiating a commercial agreement.

The Arsenal demonstration will walk through a live run on a Windows 11 virtual machine: loading the kernel driver, running the scoring agent, executing the PPID spoofing and LSASS harnesses from the evasion matrix, demonstrating the automatic kill response and the visibility state transition, and showing the failure lab promoting a sensor to DEGRADED when the queue flood scenario fires. The demonstration will also reproduce the PPL negative result to show that the tool reports what it cannot see rather than silently missing it.

7 Conclusion

Entity addresses a property of Windows telemetry that is usually treated as background noise: sensors disagree, and the disagreement is information. By running Sysmon, ETW and a kernel driver as three independent witnesses and promoting any gap or conflict to a finding, Entity detects technique classes that single sensor stacks miss by design. Its visibility model quantifies what the stack can currently see and qualifies every alert count accordingly. Its evaluation methodology, a live evasion matrix that records negative results honestly and a failure injection

lab that found real bugs, provides the kind of evidence that deployment decisions should require but rarely get.

The project is open source under the MIT License, has a kernel driver, applies automated response, and publishes both its evasion results and its health failure cases. No other open source Windows endpoint tool reviewed for this work combines all four.

References

[1] M. Russinovich and T. Garnier, "Sysmon v15.0," Microsoft Sysinternals, 2024. [Online]. Available: https://learn.microsoft.com/en-us/sysinternals/downloads/sysmon

[2] Microsoft Corporation, "PsSetCreateProcessNotifyRoutineEx routine," *Windows Driver Kit Documentation*, 2024. [Online]. Available: https://learn.microsoft.com/en-us/windows-hardware/drivers/ddi/ntddk/nf-ntddk-pssetcreateprocessnotifyroutineex

[3] Microsoft Corporation, "ObRegisterCallbacks routine," *Windows Driver Kit Documentation*, 2024. [Online]. Available: https://learn.microsoft.com/en-us/windows-hardware/drivers/ddi/wdm/nf-wdm-obregistercallbacks

[4] P. Yosifovich, A. Ionescu, M. E. Russinovich, and D. A. Solomon, *Windows Internals*, 7th ed. Microsoft Press, 2017.

[5] fireeye, "pywintrace 0.2.0," GitHub, 2019. [Online]. Available: https://github.com/fireeye/pywintrace

[6] N. Šabić, "Fibratus," commit `1fd558da6900`, GitHub, 2024. [Online]. Available: https://github.com/rabbitstack/fibratus

[7] Comodo Security Solutions, "OpenEDR," commit `30b36f203afb`, GitHub, 2024. [Online]. Available: https://github.com/ComodoSecurity/openedr

[8] V. Agarwal et al., "Velociraptor," commit `ab374970f21c`, GitHub, 2024. [Online]. Available: https://github.com/Velocidex/velociraptor

[9] Wazuh, Inc., "Wazuh," commit `a61761ef0be2`, GitHub, 2024. [Online]. Available: https://github.com/wazuh/wazuh

[10] osquery Project, "osquery," commit `8d9442011e17`, GitHub, 2024. [Online]. Available: https://github.com/osquery/osquery

[11] Elastic N.V., "Elastic Defend: Parent Process PID Spoofing," *Elastic Security Documentation*, 2024. [Online]. Available: https://www.elastic.co/guide/en/security/current/parent-process-pid-spoofing.html

[12] Microsoft Corporation, "Event Tracing for Windows," *Windows Developer Documentation*, 2024. [Online]. Available: https://learn.microsoft.com/en-us/windows/win32/etw/event-tracing-portal

[13] D. Childs, "Parent PID Spoofing," *MITRE ATT&CK*, Technique T1134.004, 2020. [Online]. Available: https://attack.mitre.org/techniques/T1134/004/